

Sardine Experiments Overview

Evolution of Input Image Handling

Initially, we used complete images for classification, but background interference and noise affected the model's focus on relevant features. To address this, we transitioned to using cropped images obtained with the YOLO fish detection model, which helped remove irrelevant backgrounds. However, for better accuracy in firmness-related classifications, we further refined our approach by using segmented images with black backgrounds extracted from the YOLO instance segmentation model for fish. Identified major reasons for classification into "Bad" class : Softness, size, cuts, damages, smell, color, detached head. This ensured the model focused solely on the fish, eliminating distractions and improving classification performance. This is especially helpful for softness based freshness classification.

Fish Segmentation YOLO Model

The current YOLO model was trained on ____ images with ____ validation images and ____ test images annotated in Roboflow. It was trained for ____ epochs and image resizing to 640,640.

Cuts & Damages Segmentation YOLO Model

The 1st step of classification is cuts and damage detection. It detects the cuts and damages that are visible.

The current YOLO model was trained on ____ images with ____ validation images and ____ test images annotated in Roboflow. It was trained for ____ epochs and image resizing to 640,640.

If it detects any cuts or damage, we classify the fish as bad otherwise it goes to the next step.

Exploratory Data Analysis (EDA)

Analyzed histograms and scatter plots of normal images and PCA components:

- Metrics included mean, median, skewness, kurtosis, and percentiles.
- Colorspaces: LBP, HOG, and GLCM features.
- **Result:** No significant insights were found.

Sardine Freshness Classification Models:

Dataset: Using segmented single fish images with a black background for training models for better color information. Because the segmented fish image from an image of a group is not that clear due to the fish being placed too close to each other that they overlap sometimes.

Version 1 (06-Sep-23 to 20-Sep-23)

➤ **Model Architecture Freshness Model Version 1 :**

In the initial version, a machine learning model employing Support Vector Machine (SVM) was utilized. The feature vector for classification was derived from the saturation channel histogram of the HSV image, as it was observed that saturation differences could be indicative of fish

freshness. The model's input consisted of cropped images, obtained through a pre-existing fish detection model, categorized into 'Good' and 'Bad' classes based on freshness.

➤ **Model Selection :**

Various machine learning models were experimented with during the development, but SVM consistently outperformed others in terms of classification accuracy. The decision to use SVM was based on its ability to effectively capture the nuances in saturation features for distinguishing between good and bad sardines.

➤ **Training Data :**

The training dataset comprised 500 "clear" images for each class ('Good' and 'Bad').

➤ **Preprocessing Steps :**

- Use of Cropped Images
- Aspect ratio

Simplification through Horizontal Orientation

- Padding to maintain aspect ratio was observed to lead to information loss, particularly when the segmented fish became smaller than the background while maintaining the aspect ratio in some images. This resulted in a reduction in the amount of fish-related information available to the model. So maintaining a consistent horizontal orientation instead.
- To address the challenges associated with aspect ratio maintenance and padding, a decision was made to maintain a consistent horizontal orientation instead. This involved rotating vertically oriented images to a horizontal orientation without the use of padding. This adjustment aimed to ensure that the orientation of the images was consistent, simplifying the learning process for the model.

- Resizing

- All images were resized to a target size of (280, 180), with the width intentionally kept greater than the height for simplicity and to preserve the horizontally oriented nature of the majority of images.
- The target size was selected to strike a balance between feature vector size and compatibility with commonly used pre-trained CNN models which is 224x224.



➤ **Initial test results :**

- The test dataset includes 500 images for each class.
- The overall accuracy is **77%** with balanced precision and recall between both the classes.

Confusion Matrix					
Predicted Labels	Good	400	100		
	Bad	131	369		
		True Labels			
		Good	Bad		

	precision	recall	f1-score	support
Good	0.75	0.80	0.78	500
Bad	0.79	0.74	0.76	500
accuracy			0.77	1000
macro avg	0.77	0.77	0.77	1000
weighted avg	0.77	0.77	0.77	1000

➤ **App testing results :**

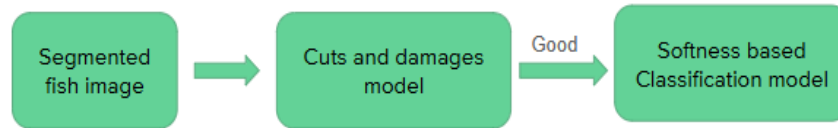
- The overall accuracy is **70%** with notable recall and precision for the “Good” class.

V1 Sardine(06-Sep-23 to 20-Sep-23)					
Actual Values			Classification Metrics Report		
Predicted Values			Precision	Recall	F1-Score
	Good	Bad			
Good	608	169	0.78	0.83	0.81
Bad	122	64	0.34	0.27	0.31
Accuracy					0.70
Macro Avg			0.56	0.55	0.56
Weighted Avg			0.68	0.70	0.69
					963

➤ **Test Link :** [test sardine.ipynb](#)

Version 2 (21-Sep-23 to 22-Oct-23)

- The cuts and damages model was integrated into the sardine freshness classification project to identify and account for physical damage in sardine images, influencing their overall classification.



- System Functionality: If the cuts and damages model predicts the presence of damage in an input image, the system immediately categorizes it as 'Bad' without further processing by the freshness classification model.

V2 Sardine(21-Sep-23 to 22-Oct-23)

Actual Values

	Good	Bad
Good	692	455
Bad	121	147

Classification Metrics Report

	Precision	Recall	F1-Score	Support
Good fish	0.60	0.85	0.71	813
Bad fish	0.55	0.24	0.34	602
Accuracy			0.59	1415
Macro Avg	0.58	0.55	0.52	1415
Weighted Avg	0.58	0.59	0.55	1415

Version 4 (20-Nov-23 to 28-Nov-23)

- **Transition to CNN Models: Freshness Model Version 2 Evolution**
 - Transitioned from SVM to Convolutional Neural Networks (CNNs) for freshness classification.
- **Dataset Expansion and Characteristics**
 - Dataset for CNN Models: Expanded the dataset to include **797** bad and **795** good segmented images with a **black background**.
 - Characteristics: Utilized **segmented** RGB images for training, maintaining the same pre-processing methods and target size (280x180x3) from Version 1.

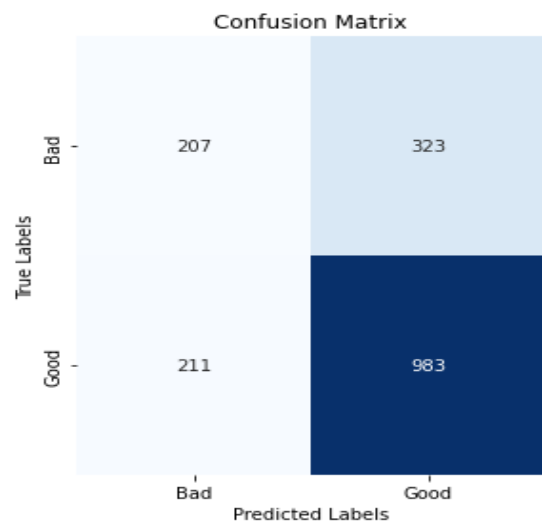
➤ **Model Architecture:**

- Base Model: Densenet201
- Image Size: 280x180x3 (RGB)
- Layers:
 - Global Average Pooling
 - 40 Neurons Dense Layer
 - 50 Neurons Dense Layer
 - 50% Dropout Layer
 - 50 Neurons Dense Layer
 - 50% Dropout Layer
 - Output Layer with Softmax Activation Function
- Loss Function: Sparse Categorical Cross Entropy
- Epochs: 15
- Batch size - 32
- Resize - (280, 180)
- Preprocessing - rescaling with /.255
- Metrics – **recall good** and **precision bad**
- Loss – Sparse Categorical cross-entropy
- Call-backs – Learning rate schedule, Early stopping

➤ Model Training:

- The results for the MobileNetV2 and DenseNet201 gave results with recall of 75%, 65% for the good class and 39%, 57% for the bad class respectively.
- The model with **DenseNet201** as base model performed better than MobileNetV2.

- **Initial test results** : The results indicate a 70% recall for the positive class (good) and a 40% recall for the negative class (bad).



➤ **App testing results :**

The overall accuracy was too low(**51%**) because we were still getting cropped images instead of segmented in the backend.

V4 Sardine(20-Nov-23 to 28-Nov-23)

Actual Values

Predicted Values

Good

Bad

Good

Bad

	Good	Bad
Good	43	52
Bad	14	26

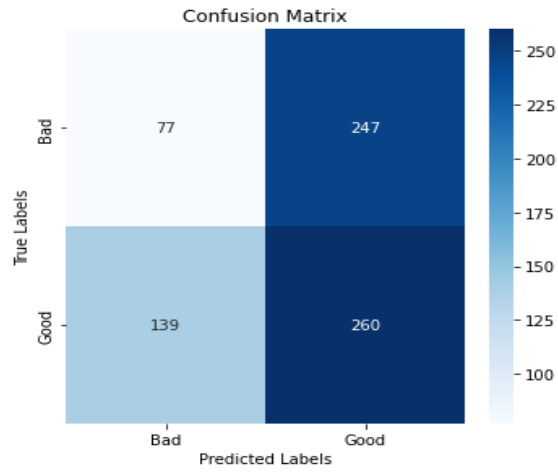
Classification Metrics Report

	Precision	Recall	F1-Score	Support
Good fish	0.45	0.75	0.57	57
Bad fish	0.65	0.33	0.44	78
Accuracy			0.51	135
Macro Avg	0.55	0.54	0.50	90
Weighted Avg	0.57	0.51	0.49	90

➤ **Test link :** [🔗 Sample test sardine v2.ipynb](#)

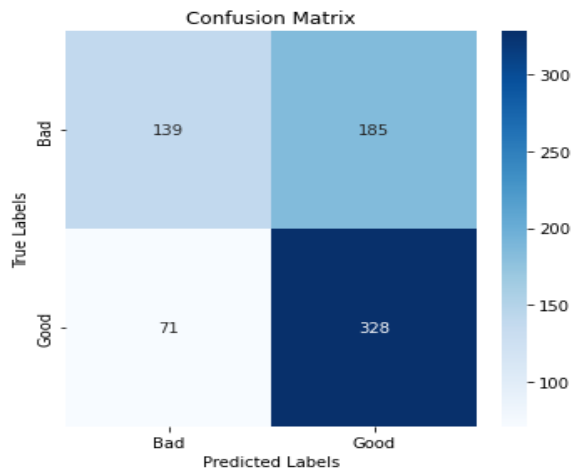
➤ **Model Comparison and Evaluation:**

- The new CNN model has already been trained on the dataset till 10-12-23. So I used a dataset after 10-12-23 for testing the model's performance.
- Conducted tests with both Version 1 and Version 2 models on different dates' data.
- Outperformed the old saturation histogram model on unseen data, exhibiting a 64% precision and 82% recall for 'Good', while the old saturation histogram model gave 51% precision and 65% recall for the 'good' class for the same dataset.



Old model results

	precision	recall	f1-score	support
Bad	0.36	0.24	0.29	324
Good	0.51	0.65	0.57	399
accuracy			0.47	723
macro avg	0.43	0.44	0.43	723
weighted avg	0.44	0.47	0.44	723



New model results

	precision	recall	f1-score	support
Bad	0.66	0.43	0.52	324
Good	0.64	0.82	0.72	399
accuracy			0.65	723
macro avg	0.65	0.63	0.62	723
weighted avg	0.65	0.65	0.63	723

Version 5 (29-Nov-23 to 30-Nov-23)

Freshness Model Version 3 Model

➤ Model Architecture:

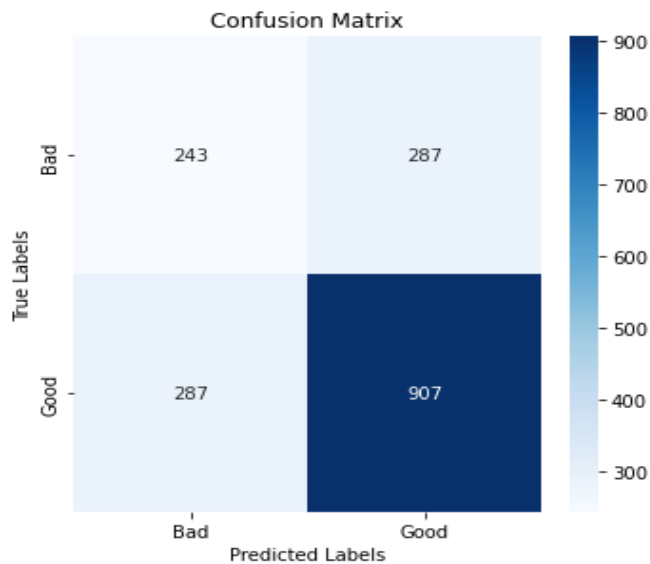
- Base Model: Densenet201
- Image Size: 280x180x3 (RGB)
- Layers:
 - Global Average Pooling
 - 40 Neurons Dense Layer
 - 50 Neurons Dense Layer
 - 20% Dropout Layer
 - 50 Neurons Dense Layer
 - 20% Dropout Layer
 - Output Layer with Softmax Activation Function
- Loss Function: Sparse Categorical Cross Entropy
- Epochs: 15
- Batch size - 32
- Resize - (280, 180)
- Preprocessing - rescaling with /.255
- Metrics – **recall bad** and **precision good**
- Call-backs – Learning rate schedule, Early stopping

➤ Training Details:

- Dataset: Same as V2, consisting of segmented sardine images.
- Pre-processing: Consistent pre-processing functions and target size as V2.

➤ Initial test results:

- Accuracy: Achieved an accuracy of **67%** with an increase in **2%** with the previous model during training.
- Recall Improvement: Noticeable improvement in recall for both 'good' and 'bad' classes with test data.



	precision	recall	f1-score	support
Bad	0.46	0.46	0.46	530
Good	0.76	0.76	0.76	1194
accuracy			0.67	1724
macro avg	0.61	0.61	0.61	1724
weighted avg	0.67	0.67	0.67	1724

➤ **App testing results:**

- The overall accuracy was too low(**48%**) because we were still getting cropped images instead of segmented in the backend.
- Precision for the 'bad' class and recall for the 'good' class were notably high.

V5 Sardine(29-Nov-23 to 30-Nov-23)

Actual Values		Classification Metrics Report			
Predicted Values		Good	Bad		
	Good	57	0	Precision	Recall
	Bad	63	2	F1-Score	Support
Good fish		1.00	0.47	0.64	120
Bad fish		0.03	1.00	0.06	2
Accuracy				0.48	122
Macro Avg		0.52	0.74	0.35	122
Weighted Avg		0.98	0.48	0.63	122

➤ **Test link:** [🔗 Sample test sardine v3.ipynb](#)

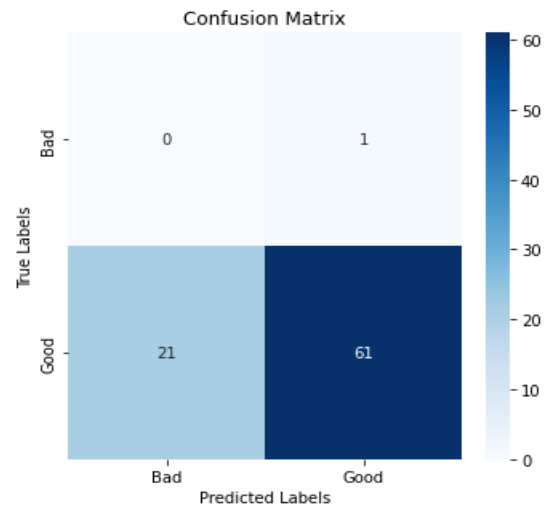
Version 7 (06-Dec-23)

➤ Reversion to Freshness Model Version 2

- **Model Comparison:** I conducted tests using both models on different dates' data in my local, and the results indicate that the previous model(V2) performed better, demonstrating notably higher precision for the good class and better overall accuracy.

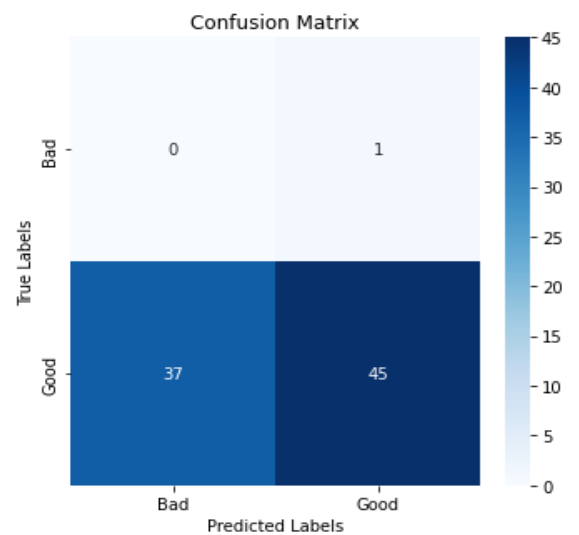
V2 Freshness model results

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	1
Good	0.98	0.74	0.85	82
accuracy			0.73	83
macro avg	0.49	0.37	0.42	83
weighted avg	0.97	0.73	0.84	83



V3 Freshness Model results

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	1
Good	0.98	0.55	0.70	82
accuracy			0.54	83
macro avg	0.49	0.27	0.35	83
weighted avg	0.97	0.54	0.69	83



Recent Version :

Data Sources

Daily data collected from S3 and manual data till Oct-8-2024.

Dataset Segmentation

- Segmented and organized “single” fish image data into ‘Good’ and ‘Bad’ folders.
- 500 images of the app testing data per class has been included in training and validation datasets as they are 3 fish “group” images.

Data Distribution

Dataset	Good	Bad
Training	5968	6010
Validation	1869	1873
Test	1712	1711
Unseen	50	51
App testing data	1423	670

Model Architecture

Layer (type)	Output Shape	Param #
densenet121 (Functional)	(None, 7, 7, 1024)	7,037,504
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1024)	0
dense (Dense)	(None, 512)	524,800
batch_normalization (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 1)	257

Modifications

- New Dataset Version: Included segmented images of group fish (500 images per class for training and validation) from app testing data in training and validation data.
- Incorporated Batch Normalization:
 - a. Significance: Normalizes outputs for faster training and better performance.
 - b. Enhanced model stability and convergence.
- Architecture:
 - a. Included Batch Normalization layers after dense layers.
 - b. Applied **L2 Regularization** in the **output layer** to prevent overfitting.
- Further Model Changes: Enabled all layers of the base model for training (fine-tuning).
- Data Augmentation: Various augmentation techniques were applied, including rotation, zoom, shift, and adjustments in brightness and color, to increase the model's robustness and generalization ability.

Experiment 4: Batch Normalization and additional fine-tuning

Framework:

- TensorFlow Version: 2.16 (model loading issue fixed).

Training Configuration:

- Continued training from saved weights (previous 50-epoch model).
- Learning Rate: Lowered to 1e-5.
- Ran for 20 epochs with data augmentation.

Results:

- Training Accuracy: 88%
- Validation Accuracy: 81%
- Test Accuracy: 83%
- Unseen Data Accuracy: 86%
- App Testing Data Accuracy: 81%

Insights: The introduction of batch normalization significantly boosted performance metrics.



➤ Model Architecture:

- Base Model: Densenet121
- Image Size: 224x224x3(RGB)
- Layers:
 - Global Average Pooling
 - 512 Neurons Dense Layer
 - Batch Normalization
 - 10% Dropout Layer
 - 256 Neurons Dense Layer

- Batch Normalization
- 10% Dropout Layer
- Output Layer with Sigmoid Activation Function
- Loss Function: Binary Cross Entropy
- Epochs: 50
- Batch size - 32
- Preprocessing - densenet preprocessing function
- Metrics – **accuracy**
- Call-backs – Learning rate schedule, Early stopping
- Data Augmentation - Done

Training Accuracy Loss Plots



Fine-tuning Training Configuration:

- Continued training from saved weights (previous 50-epoch model). This time unfreezing the first layer of the DenseNet121 model.
- Learning Rate: Lowered to $1e-5$.
- Ran for 20 epochs with data augmentation.
- **Insights:** The introduction of batch normalization significantly boosted performance metrics.

Results: Link to Confusion Matrices PPT: [\[Link\]](#)

- Training Accuracy: 88%
- Validation Accuracy: 81%
- Test Accuracy: 83%
- Unseen Data Accuracy: 86%
- App Testing Data Accuracy: 81%

Transfer Learning vs Fine-tuning:

Transfer learning initializes a model for a target task using knowledge gained from a source task, typically via pre-trained weights. It aims to transfer learned representations, often involving feature extraction or weight initialization.

Fine-tuning is a specific transfer learning technique that further adapts the pre-trained model's parameters directly on the target task's data. This optimization refines the model's representations for improved performance on the target domain.

- The key distinction lies in the degree of adaptation: transfer learning encompasses broader strategies, while fine-tuning focuses on direct parameter updates.
- Transfer learning can involve freezing layers or using the pre-trained model as a fixed feature extractor, whereas fine-tuning typically updates most or all layers.
- Both leverage pre-training to enhance generalization and accelerate training.

➤ App testing results:

- The overall accuracy was better(**73%**) because the majority of the training dataset is single fish images whereas the app testing is done with 3 fish in the image.
- Recall for the 'bad' class and recall for the 'good' class were high.

V16 Sardine(02-Dec-24 to today)

Actual Values			Classification Metrics Report				
Predicted Values			Precision	Recall	F1-Score	Support	
	Good	Bad					
	309	74	0.81	0.72	0.76	432	
			0.64	0.75	0.69	297	
	Bad	123			0.73	729	
	223						
Today : 18-Jan-25			Accuracy			0.73	729
			Macro Avg	0.73	0.73	0.73	729
			Weighted Avg	0.74	0.73	0.73	729

Fish Size based Classification

The 2nd step of classification is size based classification. So first we need to get the size. But if we directly extract the fish size in pixels from the segmented image, it may be distracting. Because not all the images are taken from the same angle and height. So sometimes when the image is captured from a low angle even a small fish looks bigger. So it is misleading.

That's why we proposed using a 15cm scale as a reference object and trained another YOLO segmentation model to detect the scale.

Dataset: Trained YOLO segmentation model on 753 images with different 15 cm scales. By isolating the measurement-specific part of the scale, the model can effectively function with any scale image, providing greater versatility and accuracy. 551 train, 151 validation and 51 test images.

Then as we know that the scale's actual length is already 15cm, after getting the segmented image of the scale, we will find the pixels_per_cm value.

Now we can use the same for the fish and convert any of its dimensions into centimeters which is a fixed unit. Now it will not be misleading.

After converting the fish height from pixels to centimeters, we found that usually sardines which are less than 13cm are considered babies. So mostly they will not be classified as Fresh to be eaten. Hence we are considering them under “bad” class only.

But we also experimented by extracting some features from the segmented fish image and the whole image in pixels. Then converting them into centimeters using the pixels_per_cm value. The features include:

- FishHeight, FishWidth, FishDiagonalLength from the segmented fish image
- FishArea, FishCircumference by applying contour detection and finding the largest contour and the area occupied by that contour
- Aspect ratio, ImageHeight/FishHeight, ImageWidth/FishWidth, ImageDiagonalLength/FishDiagonalLength, ImageArea/FishArea

Note that I used fish diagonal length as the fish height because sometimes the fish is placed diagonally in the image. So to cover those cases as well, I am using segmented fish diagonal length as height instead of height of the segmented fish image.

Then training ML classification models with these features as inputs and Good and Bad as outputs. We found that many of these features have different ranges for Good and Bad classes. So the models also performed really well.

- We balanced the dataset by selecting 160 bad and 160 good images for training, with an additional 40 images from each class for validation.
- Logistic regression outperformed the others with Train accuracy: 98%, validation accuracy: 95%, test accuracy: 94% with threshold 0.5.

We also tried manually entering the fish size and training ML regression models for predicting the fish height in centimeters so that we can convert all the dimensions into centimeters without the need of any other reference object

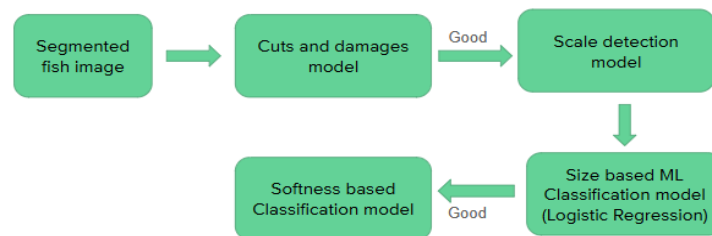
- by flattening the image array

- by extracting some features from the segmented fish image and the whole image in pixels we used in the previous experiments except that here we are leaving them in pixels.

However, the ML models' MeanAbsoluteError was more than 2.1cm. We also tried PCA for dimensionality reduction. Still the accuracy of predicting fish size in centimeters did not improve.

Scale based size prediction is performing better than this. So we are using scale based fish size prediction in the size based classification.

Integrated size model as the 2nd step with reference scale height in centimeters fixed to 13cm as the scales were not captured completely in the image. A logistic regression model was used with 0.5 threshold.



➤ App Testing results:

- The accuracy was low(51%) because the scales were not captured completely in the images.

V11 Sardine(23-Apr-24 to 10-June-24)

Actual Values			Classification Metrics Report			
Predicted Values	Good Bad		Precision	Recall	F1-Score	Support
	Good	Bad	0.42	0.85	0.56	242
	Bad	Good	0.79	0.32	0.46	428
Accuracy					0.51	670
Macro Avg			0.61	0.59	0.51	670
Weighted Avg			0.66	0.51	0.50	670

I am currently working on increasing the dataset for the same experiment of training ML classification models with the fish dimensions in centimeters and some ratios as features with balanced datasets.

These are the results of the first iteration.

Experiment	Dataset	Train data (per class)	Test data (per class)	Unseen data (per class)	Model	Test accuracy	Unseen accuracy	Test Bad f1-score	Test Good f1-score
1	data collection only	1467	368	100	SVC	93%	92%	93%	93%
					XGB	92%	94%	92%	92%
2	App testing only	162	40	50	RandomForest	77%	74%	76%	77%
					Gradient Boosting	77%	75%	76%	77%
3	combined	2086	522	100	RandomForest	92%	90%	92%	92%
					XGB	92%	93%	91%	92%

Dataset Overview:

Incorporated data collected till **December 26, 2024** along with the app testing data till **December 22, 2024**.

- **Total Data Points:** 4299 Good | 2836 Bad
 - **Data Collection Data:** 2258 Good | 1937 Bad
 - **App Testing Data:** 1321 Good | 666 Bad
 - **December Data:** 720 Good | 233 Bad
- **Train/Validation/Unseen Split:**
 - Training: 2188 per class
 - Validation: 548 per class
 - Unseen: 100 per class
- Additional **Unseen Dataset 2:** 1463 Good | 10 Bad

XGB Results:

Dataset Accuracy

Validation 82%

Unseen 1 81%

Unseen 2 85%

- **December Data Only:** 86%
- **App Testing (Aug–Dec):** 85%
- **App Testing (Till Aug):** 85%

SARDINE MODEL WORKFLOW

1. Cuts and Damages Model: The image is first passed through the cuts and damages model. If the model predicts a bad outcome, the process concludes with a 'bad' classification.

2. Size Model: If the cuts and damages model predicts good, the image is then routed to the size model. Here, the model evaluates the size of the sardine based on a predefined scale height of 15 centimeters. If the size based ML classification model predicts bad, the process concludes with a 'bad' classification.

3. Softness Model: If the size model predicts good, the image proceeds to the softness based classifying CNN model. This model assesses the softness of the sardine and provides its predictions.

Flowchart

